

Open-Book Exam Reference

Syed Zahid Abbas

Contents

1	Chapter 1: Mathematical Foundations	4
1.1	Random Variables and Expectations	4
1.2	Rules of Expectations	4
1.3	Conditional and Marginal Distributions	4
1.4	Jensen’s Inequality	4
1.5	Relationships Between Joint, Conditional, and Marginal Distributions	4
1.6	Law of Total Expectation	4
1.7	Law of Total Variance	5
1.8	KL Divergence, Entropy, Cross-Entropy, and Mutual Information	5
1.9	Maximum Likelihood Estimation (MLE)	5
1.10	Monte Carlo Estimation	5
2	Chapter 2 – Variational Autoencoders	5
2.1	Deterministic Autoencoder vs. Variational Autoencoder	5
2.2	Why Go Beyond a Deterministic Autoencoder?	6
2.3	The Basic VAE Model	6
2.4	The Intractability of the Marginal Likelihood	6
2.5	Derivation of the Evidence Lower Bound (ELBO)	7
2.6	ELBO Optimization on a 1D Toy Example	7
2.7	Encoder and Decoder Math / Reparameterization Trick	7
2.8	Sampling Failure in AE (1D Mixture Example)	8
2.9	VAE Success (1D and MNIST)	8
2.10	Entanglement and Disentanglement in Latent Variable Models	8
2.11	Inductive Bias and Why Disentanglement Does Not Come for Free	8
2.12	β -VAE for Disentanglement	9
2.13	Conditional VAE (CVAE) for Disentanglement	9
2.14	MNIST Comparison of VAE, β -VAE, and CVAE	9
2.15	Common Failure Modes in VAEs	10
2.16	Prior vs. Posterior – The Core Dynamic	10
2.17	Mathematical Goal of a VAE (Summary)	11
3	Chapter 3: Diffusion Models	11
3.1	Markov Chains – The Foundation of Diffusion	11
3.2	The Forward Diffusion Process	11
3.3	Understanding α_t , β_t and the Noise Schedule	11
3.4	Notation: Trajectories and Marginals	11
3.5	The Reverse Process Is Intractable	12
3.6	Learning the Reverse Process	12
3.7	Training the Noise-Prediction Model	12
3.8	U-Net Architecture for Images	12
3.9	Time Conditioning and Sinusoidal Embeddings	12

3.10	DDPM Sampling Algorithm	13
3.11	1D DDPM Toy Example	13
3.12	Mutual Information Decay	13
3.13	Comparing Linear vs. Cosine Schedules	13
3.14	Forward and Reverse Histograms	13
3.15	DDIM – Deterministic and Fast Sampling	13
3.16	2D Blob Denoising with a Tiny U-Net	14
3.17	Common Diffusion Families	14
4	Chapter 4: Advanced Diffusion	14
4.1	Classifier-Guided Diffusion	14
4.2	Hyperparameters in Classifier-Guided Diffusion	14
4.3	Classifier-Free Guidance (CFG)	15
4.4	SiLU (Swish) Activation Function	15
5	Chapter: 5 Flow Matching	15
5.1	General Idea and Intuition Behind Flow Matching	15
5.2	How Flow Matching Obviates Estimation Issues Faced by VAE and Diffusion	15
5.3	Intuition Behind Flow Matching	16
5.4	Rationale for Various Neural Network Architectures in Flow Matching	16
5.5	Classifier-Guided and Classifier-Free Ideas Applied to Flow Matching	16
5.6	Latent Flow Matching	16
5.7	Application of Optimal Transport to Flow Matching	16
5.8	Idea of Consistency Models	17
5.9	Why Consistency Models Are Difficult to Train	17
5.10	Design Dimensions and Variants of Flow Matching Solutions	17
5.11	Building Towards a Transport-Based View	17
5.12	Some Publishable Ideas	17
6	Chapter: 6 Transformers	17
6.1	What is an Autoregressive Model	17
6.2	How Recurrent Neural Networks (RNNs) Work	18
6.3	Long Short-Term Memory (LSTM) and Its Limitations	18
6.4	Basic Ideas Behind the Transformer Architecture	18
6.5	Byte-Level BPE Tokenization	18
6.6	Decoder-Only Transformer with Causal Self-Attention	19
6.7	Training, Evaluation, and Generation on a Simple Corpus	19
6.8	Worked Attention Example: Scores, Masking, and Softmax Weights	19
7	Chapter 7: Vision Transformers	19
7.1	Adapting Transformers to Vision	19
7.2	Patch Embedding: From Image to Sequence	19
7.3	Position Embeddings in ViT	20
7.4	The Classification Token (CLS Token)	20
7.5	Multi-Head Self-Attention in Vision	20
7.6	ViT Architecture Overview	20
7.7	Comparison with CNNs: Inductive Biases	20
7.8	Training and Data Requirements	21
7.9	Variants and Extensions	21
7.10	Summary	21

8	Chapter 8: Vision–Language Models with a Special Emphasis on CLIP	21
8.1	Dual-Encoder Contrastive (CLIP-like) Architecture	21
8.2	Fusion / Encoder–Decoder Architecture and Cross-Attention	22
8.3	Role of Paired Image–Text Data and Data Noise	22
8.4	Evaluation of VLMs: Beyond Single Benchmarks	22
8.5	The CLIP Similarity Matrix and Contrastive Loss	22
8.6	Zero-Shot Classification with CLIP	23
8.7	Cross-Modal Retrieval in CLIP	23
8.8	Joint Embedding Space and t-SNE Visualization	23
8.9	Image-Conditioned Text Generation in Fusion Models	23
8.10	Cross-Attention Mechanism in Fusion VLMs	23
8.11	Extensions Beyond CLIP	24
9	Chapter 9: Training and Adapting Vision-Language Models	24
9.1	The Adaptation Ladder for VLMs	24
9.2	Weight-Updating vs. Prompt-Only Methods	24
9.3	Multimodal Input Processing in a VLM Pipeline	25
9.4	Few-Shot Learning and RAG Without Weight Updates	25
9.5	Supervised Fine-Tuning (SFT) Objective	25
9.6	LoRA and QLoRA: Parameter-Efficient Fine-Tuning	25
9.7	Preference Data for DPO	25
9.8	DPO Loss and Intuition	26
9.9	KTO and Alternatives	26
9.10	Online Reinforcement Learning for VLM Alignment	26
9.11	Key Reinforcement Learning Concepts in VLM Alignment	26
9.12	Trade-Offs in Choosing an Adaptation Method	26
10	Chapter 10: Quantization Techniques for Deep Learning and Generative Models	27
10.1	Motivation for Quantization	27
10.2	Three Phases of Quantization	27
10.3	Standard Quantization Formula and Parameters	27
10.4	The Scale–Range Trade-off	27
10.5	Per-Tensor, Per-Channel, and Group-wise Quantization	28
10.6	Quantization-Aware Training (QAT)	28
10.7	Post-Training Quantization (PTQ)	28
10.8	Second-Order Sensitivity and the Role of the Hessian	28
10.9	Modern Transformer and LLM Quantization Methods	28
10.10	Quantization for Generative Models	29
10.11	Comparative Challenges Across Model Families	29

1 Chapter 1: Mathematical Foundations

1.1 Random Variables and Expectations

A random variable X is a variable whose possible values are numerical outcomes of a random phenomenon, governed by a probability distribution $p(x)$. The expectation $\mathbb{E}[X]$ is the weighted average of all possible values: for discrete X , $\mathbb{E}[X] = \sum_x x p(x)$; for continuous X , $\mathbb{E}[X] = \int x p(x) dx$. More generally, for any function f , $\mathbb{E}[f(X)] = \sum_x f(x)p(x)$ (or the integral analogue). The expectation captures the central tendency of the distribution and is the fundamental building block for variance, covariance, and many probabilistic quantities.

1.2 Rules of Expectations

Expectation is a linear operator: for constants a, b, c and random variables X, Y ,

$$\mathbb{E}[aX + bY + c] = a \mathbb{E}[X] + b \mathbb{E}[Y] + c,$$

even when X and Y are dependent. Consequently, the expectation of a sum equals the sum of expectations, and the expectation of a constant is the constant itself. These rules simplify many derivations in generative modelling and learning theory, allowing complex expressions to be decomposed into manageable parts without requiring independence assumptions.

1.3 Conditional and Marginal Distributions

A marginal distribution $p(x)$ isolates a single variable from a joint distribution $p(x, y)$ by summing (or integrating) over the other variable: $p(x) = \sum_y p(x, y)$. It describes the behaviour of X alone. Conditional distributions, such as $p(x | y) = p(x, y)/p(y)$, describe how X behaves given a fixed value of Y . Conditional expectations $\mathbb{E}[X | Y = y]$ compute the average of X within the subset defined by $Y = y$, and the function $\mathbb{E}[X | Y]$ is itself a random variable depending on Y .

1.4 Jensen's Inequality

Jensen's inequality relates expectations of convex (or concave) functions to the function evaluated at the expectation. For a convex function ϕ , $\phi(\mathbb{E}[X]) \leq \mathbb{E}[\phi(X)]$; for a concave function, the inequality reverses. This result is essential for bounding quantities such as log-likelihoods, deriving the Evidence Lower Bound (ELBO) in variational inference, and understanding why Monte Carlo estimates of log-probabilities are biased.

1.5 Relationships Between Joint, Conditional, and Marginal Distributions

The joint distribution $p(x, y)$ contains all information about (X, Y) and can be factorized in two symmetric ways: $p(x, y) = p(x | y)p(y) = p(y | x)p(x)$. Rearranging gives Bayes' rule, $p(y | x) = p(x | y)p(y)/p(x)$. Marginal distributions are obtained by summing out the other variable, and conditional distributions are obtained by renormalizing a slice of the joint distribution. These relationships form the backbone of probabilistic reasoning, enabling model inversion, parameter estimation, and generative sampling.

1.6 Law of Total Expectation

The law of total expectation states that the overall expectation of X is a weighted average of its conditional expectations:

$$\mathbb{E}[X] = \sum_y \mathbb{E}[X | Y = y] p(y).$$

This decomposition is widely used in latent-variable models: the average over all data points can be understood as the average of group-specific means, weighted by group sizes.

1.7 Law of Total Variance

The variance of X can be decomposed into two parts: the average of the conditional variances plus the variance of the conditional means,

$$\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y]).$$

The first term captures within-group variability, the second term captures between-group variability. This identity is useful for understanding uncertainty in hierarchical models and for diagnosing how much variation is explained by latent factors.

1.8 KL Divergence, Entropy, Cross-Entropy, and Mutual Information

Entropy $H(P) = -\sum_x p(x) \log p(x)$ measures the inherent uncertainty in a distribution P . Cross-entropy $H(P, Q) = -\sum_x p(x) \log q(x)$ measures the average surprise when data from P is encoded using a model Q . The Kullback–Leibler divergence $\text{KL}(P\|Q) = \sum_x p(x) \log \frac{p(x)}{q(x)}$ quantifies the inefficiency of using Q instead of P , satisfying $H(P, Q) = H(P) + \text{KL}(P\|Q)$. Mutual information $I(X; Y) = \sum_{x,y} p(x, y) \log \frac{p(x, y)}{p(x)p(y)}$ quantifies how much knowing one variable reduces uncertainty about the other. These information-theoretic quantities are central to training objectives, regularisation, and evaluation in generative modelling.

1.9 Maximum Likelihood Estimation (MLE)

Maximum likelihood estimation selects model parameters θ that maximise the probability (likelihood) of the observed data under the model. For independent and identically distributed data $\{x_i\}_{i=1}^n$, the likelihood is $\mathcal{L}(\theta) = \prod_i p(x_i | \theta)$. In practice, the log-likelihood $\ell(\theta) = \sum_i \log p(x_i | \theta)$ is maximised because it converts products into sums, avoids numerical underflow, and directly connects to cross-entropy and KL divergence. MLE is the foundational principle behind learning in many generative models—from autoregressive models to VAEs and diffusion models—and it corresponds to minimising the cross-entropy between the empirical data distribution and the model.

1.10 Monte Carlo Estimation

Monte Carlo estimation approximates expectations by averaging over random samples: $\mathbb{E}[f(X)] \approx \frac{1}{N} \sum_{i=1}^N f(x_i)$, with $x_i \sim p(x)$. The approximation converges to the true expectation by the law of large numbers, with an error that decreases as $1/\sqrt{N}$. Monte Carlo methods are indispensable when exact integration is intractable—for example, in computing marginal likelihoods, estimating gradients for stochastic optimisation, or sampling from complex distributions in generative models. The trade-off is that high accuracy requires many samples, which can be computationally expensive.

2 Chapter 2 – Variational Autoencoders

2.1 Deterministic Autoencoder vs. Variational Autoencoder

A deterministic autoencoder learns a mapping $x \rightarrow z \rightarrow \hat{x}$, where the encoder f_ϕ produces a single code and the decoder g_θ reconstructs the input. The training objective is typically a reconstruction loss such as

$$\min_{\phi, \theta} \mathbb{E}_{x \sim p_{\text{data}}} \|x - g_\theta(f_\phi(x))\|^2.$$

A variational autoencoder (VAE) replaces the deterministic encoder with a probabilistic one: the encoder outputs the parameters of a distribution

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x))).$$

Concretely, it produces $\mu_\phi(x)$ and $\log \sigma_\phi^2(x)$. From this distribution we sample a latent vector z using the reparameterization trick

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I),$$

and then decode it through a probabilistic decoder $p_\theta(x|z)$. This turns the AE into a principled generative model that defines a full probability distribution over data.

2.2 Why Go Beyond a Deterministic Autoencoder?

A deterministic AE is excellent for compression and reconstruction, but it is *not* a generative model. It defines no explicit likelihood $p_\theta(x)$ or $p_\theta(x|z)$ with a proper prior $p(z)$. The latent space is unconstrained: the encoder is free to place codes anywhere as long as decoding works. As a result, the set of training codes $\{z_i\}$ can form clusters with large empty regions (“holes”) in between. There is no guarantee that nearby latent points decode to valid data, and sampling from a standard Gaussian $z \sim \mathcal{N}(0, I)$ typically lands in these holes, producing nonsense outputs. A VAE remedies this by making z probabilistic and regularizing the latent distribution to stay close to a chosen prior (usually $\mathcal{N}(0, I)$), so that random samples almost surely fall on the data manifold. The core distinction is: an AE puts training points somewhere in z -space and reconstructs them; a VAE puts them somewhere near a known distribution so that random samples are meaningful.

2.3 The Basic VAE Model

The VAE assumes a simple prior over latent variables: $p(z) = \mathcal{N}(0, I)$. The decoder is a generative model $p_\theta(x|z)$; for binary images a per-pixel Bernoulli distribution is used (binary cross-entropy loss), while for real-valued data a Gaussian likelihood with fixed variance is common (mean squared error corresponds to a Gaussian with constant variance). The encoder approximates the intractable true posterior $p_\theta(z|x)$ with

$$q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \text{diag}(\sigma_\phi^2(x))).$$

The overall generative process for new data is: sample $z \sim p(z)$, then sample $x \sim p_\theta(x|z)$.

2.4 The Intractability of the Marginal Likelihood

The probability of a data point under the model is

$$p_\theta(x) = \int p_\theta(x|z) p(z) dz.$$

Because the decoder is a nonlinear neural network and the latent space is typically high-dimensional (e.g. $\mathbb{R}^{16}$ to $\mathbb{R}^{256}$), this integral has no closed form. A naive Monte Carlo estimate

$$p(x) \approx \frac{1}{K} \sum_{k=1}^K p(x|z_k), \quad z_k \sim p(z)$$

can be computed, but it suffers from high variance: most prior samples yield a tiny $p(x|z_k)$, leading to noisy estimates of $\log p(x)$. The chapter demonstrates this with a 1D toy model where $p(z) = \mathcal{N}(0, 1)$, the decoder is $x|z \sim \mathcal{N}(f(z), \sigma_x^2)$ with $f(z) = z^3$ and $\sigma_x = 0.35$. Fixing $x_0 = 0.7$, the true $\log p(x_0) \approx -1.6725$ is computed by numerical integration. Monte Carlo estimates with small K show high variance and skew. In high dimensions this becomes completely impractical for training. Therefore, the VAE optimizes a tractable lower bound of $\log p_\theta(x)$ instead.

2.5 Derivation of the Evidence Lower Bound (ELBO)

Start from the log marginal likelihood:

$$\log p_\theta(x) = \log \int p_\theta(x|z) p(z) dz.$$

Multiply and divide by an arbitrary distribution $q_\phi(z|x)$ that is non-zero where the integrand is non-zero:

$$\log p_\theta(x) = \log \int q_\phi(z|x) \frac{p_\theta(x|z) p(z)}{q_\phi(z|x)} dz = \log \mathbb{E}_{q_\phi(z|x)} \left[\frac{p_\theta(x|z) p(z)}{q_\phi(z|x)} \right].$$

The term inside the expectation, $Y(z) = \frac{p_\theta(x|z) p(z)}{q_\phi(z|x)}$, is a positive random variable. Since $\log(\cdot)$ is concave on $(0, \infty)$, Jensen's inequality gives $\log \mathbb{E}[Y] \geq \mathbb{E}[\log Y]$. Substituting yields

$$\log p_\theta(x) \geq \mathbb{E}_{q_\phi(z|x)} \left[\log \frac{p_\theta(x|z) p(z)}{q_\phi(z|x)} \right].$$

Expanding the logarithm and regrouping gives the standard ELBO:

$$\mathcal{L}_{\text{ELBO}}(x) = \mathbb{E}_{q_\phi(z|x)} [\log p_\theta(x|z)] - \text{KL}(q_\phi(z|x) \| p(z)).$$

This lower bound is always $\leq \log p_\theta(x)$, and equality holds exactly when $q_\phi(z|x) = p_\theta(z|x)$, i.e. when the approximate posterior matches the true posterior. The KL divergence for diagonal Gaussians $q_\phi(z|x) = \mathcal{N}(\mu, \text{diag}(\sigma^2))$ and $p(z) = \mathcal{N}(0, I)$ can be computed analytically:

$$\text{KL} = \frac{1}{2} \sum_j (\sigma_j^2 + \mu_j^2 - 1 - \log \sigma_j^2).$$

2.6 ELBO Optimization on a 1D Toy Example

The chapter illustrates the ELBO maximization on the same 1D toy model with $x_0 = 0.7$. The variational family is assumed to be a Gaussian $q(z|x_0) = \mathcal{N}(\mu_q, \sigma_q^2)$. The ELBO is estimated by Monte Carlo sampling from $q(z|x_0)$ (4096 samples), and the parameters μ_q and $\log \sigma_q$ are optimized using finite-difference gradient ascent (learning rate 0.2, 80 steps). The true $\log p(x_0) = -1.67$, but the best ELBO achieved is around -36.28 , with final parameters $\mu_q = 0.0, \sigma_q = 1.0$. This huge gap illustrates that a simple variational family may be unable to match the true posterior, which in this case is complex because the integrand $p(x_0|z)p(z)$ is multimodal. The gap equals $\text{KL}(q(z|x_0) \| p_\theta(z|x_0))$.

2.7 Encoder and Decoder Math / Reparameterization Trick

The encoder network (parameters ϕ) outputs $\mu_\phi(x)$ and $\log \sigma_\phi^2(x)$, defining $q_\phi(z|x)$. The decoder network (parameters θ) outputs the parameters of $p_\theta(x|z)$, e.g. for a Bernoulli decoder a vector of probabilities $\hat{x}(z) \in [0, 1]^{784}$ for MNIST. To train the model with gradient descent, we need gradients of the expectation term w.r.t. ϕ . Naively sampling $z \sim \mathcal{N}(\mu_\phi(x), \sigma_\phi^2(x))$ does not allow gradients to flow because it is a stochastic operation. The reparameterization trick rewrites the sample as

$$z = \mu_\phi(x) + \sigma_\phi(x) \odot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I).$$

Now ϵ is a fixed source of randomness independent of ϕ , and the dependence on ϕ is entirely deterministic. This allows us to move the gradient inside the expectation:

$$\nabla_\phi \mathbb{E}_{q_\phi(z|x)} [f(z)] = \mathbb{E}_{\epsilon \sim \mathcal{N}(0, I)} [\nabla_\phi f(\mu_\phi(x) + \sigma_\phi(x) \odot \epsilon)].$$

This yields low-variance, unbiased gradient estimates and is the key reason VAEs are trainable. Without it, one would have to resort to high-variance score-function estimators. The trick works for any continuous distribution that can be expressed as a deterministic transformation of a fixed noise source, but Gaussians are especially convenient because they admit a simple affine transformation and the KL to the prior has a closed form.

2.8 Sampling Failure in AE (1D Mixture Example)

The chapter generates a 1D dataset that is a mixture of two components: $x_1 \sim \text{Unif}(-2.0, -1.6)$ and $x_2 \sim \text{Unif}(2.0, 2.6)$. A standard 1D AE with latent dimension 1 is trained (hidden size 64). The reconstruction plot shows good reconstruction, but the latent space (codes z plotted against x) reveals that the two clusters are mapped to disjoint intervals. When we sample $z \sim \mathcal{N}(0, 1)$ and decode, the generated samples completely fail — they are concentrated in narrow ranges far from the true data distribution. This is a direct demonstration that an AE’s latent space does not support generative sampling.

2.9 VAE Success (1D and MNIST)

A VAE with latent dimension 1 is trained on the same 1D mixture data ($\beta = 1.0$). The training curves show the reconstruction loss and KL converging. The plot of $\mu(x)$ vs. x shows a smooth, well-behaved mapping that respects the two modes. When we sample $z \sim \mathcal{N}(0, 1)$ and decode, the histogram of generated x closely matches the true data distribution. This confirms the VAE’s ability to regularize the latent space for sampling. Similarly, a VAE is trained on MNIST images (flattened to 784 pixels, Bernoulli likelihood, latent dimension 16, hidden size 400, 10 epochs). The training curves (loss, reconstruction BCE, KL) are plotted. Reconstructions are sharp, and samples drawn from $z \sim \mathcal{N}(0, I)$ produce recognizable digits. A latent traversal (varying one dimension of z while keeping others fixed) shows a smooth transition in the generated digit style, indicating that the latent manifold captures meaningful variation.

2.10 Entanglement and Disentanglement in Latent Variable Models

A representation is **entangled** when each latent dimension mixes multiple independent generative factors (e.g. digit identity, thickness, rotation). Formally, if true factors $s_1, \dots, s_k$ generate the data, an entangled z is such that changing one z_j affects several factors. Entanglement occurs naturally in standard VAEs because the ELBO only optimises reconstruction and KL to an isotropic prior. It is invariant under any orthogonal rotation of the latent space; there is no pressure to align dimensions with semantic factors. **Disentanglement** means that each latent dimension controls approximately one independent factor, enabling controllable generation and interpretability. Ideal disentanglement would have e.g. $z_1 \leftrightarrow$ digit identity, $z_2 \leftrightarrow$ stroke width, etc. Because the model has no built-in bias toward factorised semantics, disentanglement must be imposed through inductive biases; it is not automatically discovered.

2.11 Inductive Bias and Why Disentanglement Does Not Come for Free

An inductive bias is any assumption built into a model that restricts the set of solutions it can learn. A standard VAE already has biases: Gaussian prior, factorised prior $p(z) = \prod_i p(z_i)$, and smooth neural architectures. However, these are distributional and structural, not semantic. Crucially, the ELBO is invariant under latent rotations: if R is an orthogonal matrix, $z' = Rz$ has the same prior density, and the decoder can adjust to give identical reconstruction. Thus the model has no preference for axis-aligned, semantically meaningful coordinates. To encourage disentanglement, stronger biases must be added: statistical (e.g. penalising total correlation), architectural (e.g. separate subnetworks per factor), objective-based (β -VAE), or supervision-based (CVAE). There is an inherent trade-off: more inductive bias yields more interpretable,

better disentangled latents but often at the cost of reconstruction fidelity or flexibility. A fundamental theoretical result is that disentanglement is not identifiable from data alone; without inductive bias or supervision, infinitely many entangled representations explain the data equally well.

2.12 β -VAE for Disentanglement

The β -VAE modifies the ELBO by scaling the KL term with a coefficient $\beta > 1$:

$$\mathcal{L}_\beta = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \beta \text{KL}(q_\phi(z|x) \parallel p(z)).$$

Increasing β punishes the latent space for carrying too much information or having correlated dimensions. It forces the model to compress more aggressively, ideally using each dimension for a single dominant factor. The trade-off is deliberately worse reconstruction (blurrier outputs) in exchange for a cleaner, more factorised latent structure.

The chapter demonstrates this with a synthetic dataset: 1D observations $x = s \cdot a + \text{noise}$, where $s \in \{-1, +1\}$ is a discrete sign factor and $a \in [0.2, 2.0]$ is a continuous amplitude factor. A VAE with a 2D latent space (z_1, z_2) is trained with $\beta = 1$ and $\beta = 4$. The latent means $\mu(x)$ are colored by s and by a . For $\beta = 1$, one latent dimension dominates, and both information about s and a are mixed. With $\beta = 4$, both latent dimensions start to carry variation, but correlations show both z_1 and z_2 are highly correlated with the sign factor s ($\text{corr} \sim 0.91$ and ~ 0.88), and almost uncorrelated with amplitude a . Thus the model dropped the weaker factor (amplitude) and redundantly encoded the dominant factor (sign) across both dimensions. This highlights that β -VAE does *not* guarantee perfect disentanglement; it can instead ignore subtle factors or duplicate the dominant one. True clean separation (one dimension strongly correlated with s , the other with a) was not observed.

2.13 Conditional VAE (CVAE) for Disentanglement

A CVAE reduces entanglement by providing known factors explicitly as conditioning. Instead of modelling $p(x)$, it models $p(x|y)$ where y is a label or attribute (e.g. digit identity). Both encoder and decoder are conditioned on y : the encoder is $q_\phi(z|x, y)$ and the decoder is $p_\theta(x|z, y)$. The prior remains $p(z) = \mathcal{N}(0, I)$. The CVAE ELBO becomes:

$$\mathcal{L}_{\text{CVAE}}(x, y) = \mathbb{E}_{q_\phi(z|x, y)}[\log p_\theta(x|z, y)] - \text{KL}(q_\phi(z|x, y) \parallel p(z)).$$

Because y carries the known discrete/continuous factor, the latent z only needs to capture the residual unknown variation. This naturally separates concerns: one component (the condition) is interpretable, the other (latent) captures subtle styles.

The 1D example uses the same dataset $x = s \cdot a + \epsilon$, but now the condition $y = s$ (the sign) is given as a one-hot vector. The CVAE with a 1D latent is trained with $\beta = 1$ and $\beta = 4$. The latent $\mu(x, y)$ plotted against amplitude a and colored by sign shows nearly linear dependence on a with only a small sign-dependent bias. Correlations: for $\beta = 4$, $\text{corr}(z, a) \approx 0.885$ and $\text{corr}(z, s) \approx -0.128$. This is close to the ideal: z captures amplitude, and y captures sign. Latent traversal with fixed y confirms that changing z smoothly scales the output magnitude, while switching the sign condition flips the sign. This demonstrates successful conditional disentanglement: providing one factor as a label lets the latent space automatically align with the remaining independent factor.

2.14 MNIST Comparison of VAE, β -VAE, and CVAE

The chapter concludes with an MNIST implementation comparing the three models. For all, the decoder uses a Bernoulli likelihood (binary cross-entropy). The standard VAE ($\beta = 1$) learns

to reconstruct and generate digits but entangles digit identity and style. The β -VAE ($\beta > 1$) applies a stronger KL penalty, leading to more factorised latent dimensions (more interpretable traversals) but often blurrier reconstructions. The CVAE explicitly conditions on the one-hot digit label y : the encoder concatenates (x, y) and outputs $\mu, \log \sigma$; the decoder concatenates (z, y) to produce digit probabilities. Conditioned sampling becomes possible: choose y (e.g. “7”), sample $z \sim \mathcal{N}(0, I)$, and decode to get a specific digit. This removes identity from the latent bottleneck, so z controls only intra-digit style variation, greatly reducing entanglement. The training curves for all models show the typical trade-off between reconstruction and KL. The parallel summary table in the lecture notes highlights that VAE is best for general generative modeling, β -VAE for disentanglement/interpretability, and CVAE for label-controlled generation.

2.15 Common Failure Modes in VAEs

1. **Posterior collapse:** The KL term goes to zero, and $q_\phi(z|x) \approx p(z)$ for all x . The latent variables carry no information about the input, and the decoder simply learns to reconstruct from its own biases. This often happens when the decoder is too powerful or the KL weight is too high too early.
2. **Over-regularization (β too large):** In a β -VAE, if β is set excessively high, the pressure to match the prior overwhelms reconstruction. The latent space is restricted so much that the model cannot capture essential details, leading to blurry or distorted outputs.
3. **Blurry outputs (likelihood mismatch):** Using a Gaussian likelihood with fixed variance (MSE loss) encourages the decoder to output the pixel-wise mean of all plausible reconstructions. If multiple sharp outputs are possible, the safe prediction is their average, which looks blurry. Binary cross-entropy partly mitigates this for near-binary images.
4. **ELBO tightness gap:** Even when the ELBO improves, sample quality can stagnate if the approximate posterior is too simple (e.g. diagonal Gaussian) to match the true posterior. The bound becomes loose; the model finds a workaround to increase the ELBO without actually becoming a better generator. This is recognised when reconstructions are sharp but prior samples are poor. The gap equals $\text{KL}(q_\phi(z|x) \parallel p_\theta(z|x))$.
5. **Prior-posterior mismatch:** A healthy VAE has each $q_\phi(z|x)$ different from the prior but stays within the prior’s support, and the aggregate posterior $\int q_\phi(z|x)p_{\text{data}}(x)dx$ approximately matches $p(z)$. When this fails, prior samples land in untrained regions.

2.16 Prior vs. Posterior – The Core Dynamic

The prior $p(z) = \mathcal{N}(0, I)$ defines where we sample from at generation time and imposes a soft geometric constraint on the latent space. The approximate posterior $q_\phi(z|x)$ is the encoder’s belief about which latent codes are likely for a given input. The KL term $\text{KL}(q_\phi(z|x) \parallel p(z))$ does **not** force every posterior to be identical to the prior; rather, it ensures that individual latent codes do not drift arbitrarily far from the origin and that, when averaged over the entire dataset, the aggregate distribution of codes matches the prior. This allows each data point to have a distinct, informative latent representation while maintaining a globally connected, sampleable space. A useful analogy: the prior is the city boundary; the posterior is each house’s address. A healthy VAE has houses all inside the city but at different addresses. Posterior collapse means every house is at the same address (usually the city center). Over-regularization means houses are forced so close to the center that they lose their identities.

2.17 Mathematical Goal of a VAE (Summary)

The ultimate objective is to learn a generative model such that the marginal distribution

$$p_\theta(x) = \int p_\theta(x|z) p(z) dz$$

approximates the true data distribution $p_{\text{data}}(x)$. Ideally we would minimize $\text{KL}(p_{\text{data}} \| p_\theta)$, but the integral is intractable. Instead, the VAE introduces an approximate posterior $q_\phi(z|x)$ and maximizes the ELBO, which indirectly pushes $p_\theta(x)$ toward $p_{\text{data}}(x)$. At generation time, we discard the encoder, sample $z \sim p(z)$, and then sample $x \sim p_\theta(x|z)$, expecting the resulting distribution to match the empirical data distribution. Variants like β -VAE and CVAE modify this objective to achieve additional goals like disentanglement or conditional generation while still optimising a lower bound on (conditional) likelihood.

3 Chapter 3: Diffusion Models

3.1 Markov Chains – The Foundation of Diffusion

A Markov chain is a sequence of random variables $x_0, x_1, \dots, x_T$ satisfying the Markov property: the future depends only on the present, not on the past. Formally, $p(x_t | x_{t-1}, x_{t-2}, \dots, x_0) = p(x_t | x_{t-1})$. The joint distribution factorises as $p(x_{0:T}) = p(x_0) \prod_{t=1}^T p(x_t | x_{t-1})$. For diffusion models, we define a forward Markov chain that gradually destroys the data by adding Gaussian noise, and then we learn a reverse Markov chain that removes the noise step by step.

3.2 The Forward Diffusion Process

The forward (noising) process is a Gaussian Markov chain defined by the transition $q(x_t | x_{t-1}) = \mathcal{N}(x_t; \sqrt{\alpha_t} x_{t-1}, (1 - \alpha_t)I)$. Equivalently, sampling can be written as $x_t = \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon$, with $\epsilon \sim \mathcal{N}(0, I)$. The full forward chain is $q(x_{1:T} | x_0) = \prod_{t=1}^T q(x_t | x_{t-1})$. Because each step adds a small amount of noise, after many steps the distribution approaches a standard Gaussian: $x_T \approx \mathcal{N}(0, I)$. This forward process requires no learning – it is entirely defined by the chosen schedule.

3.3 Understanding α_t , β_t and the Noise Schedule

Introduce $\beta_t = 1 - \alpha_t$; then β_t is the variance of the newly injected noise at step t , while $\sqrt{\alpha_t}$ shrinks the previous signal. The cumulative product $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$ gives the fraction of the original signal that survives after t steps. A key closed-form result is the marginal distribution $q(x_t | x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t} x_0, (1 - \bar{\alpha}_t)I)$. The sequence $\{\beta_t\}_{t=1}^T$ is called the noise schedule; it controls how quickly information is destroyed. Common choices are linear schedules (β_t increasing linearly from 10^{-4} to 0.02) and cosine schedules (where $\bar{\alpha}_t$ is defined via $\cos^2$ to give a smoother signal-to-noise ratio decay). The SNR at time t is $\bar{\alpha}_t/(1 - \bar{\alpha}_t)$; a well-tuned schedule balances the difficulty of denoising across all timesteps.

3.4 Notation: Trajectories and Marginals

The notation $x_{1:T}$ means the sequence $(x_1, x_2, \dots, x_T)$, a full trajectory. The joint distribution $q(x_{1:T} | x_0)$ assigns probability to the entire noisy path. The marginal distribution $q(x_t | x_0)$ integrates out all intermediate states and represents the distribution of the state at time t alone. The closed-form marginal is a direct consequence of the Markov and Gaussian properties; it lets us sample x_t directly from x_0 without simulating all intermediate steps.

3.5 The Reverse Process Is Intractable

The true reverse transition is given by Bayes' rule: $q(x_{t-1} | x_t) = \frac{q(x_t | x_{t-1}) q(x_{t-1})}{q(x_t)}$. Although the forward transition $q(x_t | x_{t-1})$ is known and Gaussian, the unconditional marginals $q(x_{t-1})$ and $q(x_t)$ depend on the unknown data distribution, making $q(x_{t-1} | x_t)$ intractable. Conditioned on the original clean data x_0 , however, the reverse step becomes a tractable Gaussian: $q(x_{t-1} | x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta}_t I)$. At generation time x_0 is not available, so we cannot use this posterior directly. Instead, we learn a parametric approximation.

3.6 Learning the Reverse Process

Diffusion models define a learned reverse Markov chain $p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t)$, with $p(x_T) = \mathcal{N}(0, I)$. Each reverse conditional is modelled as a Gaussian: $p_\theta(x_{t-1} | x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_t)$, where the variance Σ_t is often fixed. The neural network predicts the noise ϵ that was added to turn x_0 into x_t ; that is, it learns $\epsilon_\theta(x_t, t) \approx \epsilon$. From this noise prediction, we can compute the reverse mean μ_θ using the formula

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(x_t, t) \right).$$

3.7 Training the Noise-Prediction Model

Training proceeds by sampling clean data $x_0 \sim q_{\text{data}}$, a random timestep $t \sim \text{Uniform}\{1, \dots, T\}$, and standard Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$. The noisy sample is constructed as $x_t = \sqrt{\alpha_t} x_0 + \sqrt{1 - \alpha_t} \epsilon$. The network outputs a predicted noise $\epsilon_\theta(x_t, t)$, and we minimise the simple mean-squared error loss

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{x_0, t, \epsilon} \|\epsilon - \epsilon_\theta(x_t, t)\|^2.$$

This objective is a simplified version of the variational lower bound and directly encourages the model to learn the reverse denoising direction.

3.8 U-Net Architecture for Images

For image data, the network is typically a U-Net. A U-Net consists of a downsampling path (encoder), a bottleneck, and an upsampling path (decoder), with skip connections between matching resolutions. This design preserves fine spatial details while capturing global context. The network takes as input a noisy image x_t and must output a noise-image of the same size, making the U-Net ideal. Without skip connections, high-frequency details would be lost, which is critical for pixel-accurate denoising.

3.9 Time Conditioning and Sinusoidal Embeddings

The network must know the current noise level to denoise appropriately. The integer timestep t is mapped to a high-dimensional vector via sinusoidal embeddings:

$$\text{emb}(t)_{2k} = \sin(t \cdot \omega_k), \quad \text{emb}(t)_{2k+1} = \cos(t \cdot \omega_k),$$

with frequencies $\omega_k = 10000^{-2k/d}$ (where d is the embedding dimension). This representation encodes relative time relationships; nearby timesteps have similar embeddings, while distant ones are well separated. The time embedding is injected into the network, typically by adding it to feature maps inside residual blocks (a simple form of FiLM conditioning). A common activation function is SiLU (Swish), defined as $\text{SiLU}(x) = x \cdot \sigma(x) = x/(1 + e^{-x})$, which is smooth and often works better than ReLU in diffusion models.

3.10 DDPM Sampling Algorithm

Generation starts from pure noise $x_T \sim \mathcal{N}(0, I)$ and iterates backward for $t = T - 1, \dots, 0$:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z, \quad z \sim \mathcal{N}(0, I),$$

where σ_t is often chosen as $\sqrt{\beta_t}$ (or a learned posterior variance). The first term is the predicted mean, and the second injects random noise to match the stochastic reverse variance. At $t = 0$, no noise is added, yielding the final clean sample.

3.11 1D DDPM Toy Example

A scalar DDPM is demonstrated on a 1D mixture distribution of two Gaussian bumps. A tiny MLP with sinusoidal time embedding predicts ϵ . After 2000 training iterations, the reverse chain starts from $x_T \sim \mathcal{N}(0, 1)$ and, after 200 steps, reproduces a sample distribution whose histogram matches the original two-bump data. Plots of the forward trajectory (a single scalar evolving over time) and reverse trajectory show that the noise is gradually removed, and the final value lies in one of the two data clusters.

3.12 Mutual Information Decay

The forward process destroys information about x_0 . This can be quantified empirically by estimating the mutual information $I(X_0; X_t)$. Using histogram-based estimation on the two-bump dataset, the MI starts high at $t = 0$ and rapidly drops to near zero by the end of the forward chain. This confirms that x_t becomes approximately independent of the original sample, justifying the choice of $p(x_T) = \mathcal{N}(0, I)$.

3.13 Comparing Linear vs. Cosine Schedules

The chapter compares linear and cosine noise schedules. The linear schedule increases β_t uniformly, leading to a roughly exponential decay of $\bar{\alpha}_t$. The cosine schedule defines $\bar{\alpha}_t = \cos^2\left(\frac{(t/T)+s}{1+s} \frac{\pi}{2}\right)$ and then derives β_t from consecutive $\bar{\alpha}_t$ ratios. Plots of $\bar{\alpha}_t$ vs. t , SNR vs. t , and empirical mutual information show that the cosine schedule preserves SNR longer before the final rapid drop, which often yields better sample quality and more stable training.

3.14 Forward and Reverse Histograms

Visualising the histogram of x_t at various timesteps makes the Markov chain intuition concrete. For the forward process, the bimodal data density smooths out and approaches a standard normal as t increases. For the reverse process, starting from Gaussian noise, the histogram gradually splits back into two peaks, matching the true data distribution. This directly illustrates the denoising capability of the trained reverse chain.

3.15 DDIM – Deterministic and Fast Sampling

DDIM reuses the same trained noise predictor but changes the reverse update. It introduces a parameter $\eta \in [0, 1]$ and defines

$$x_{t-1} = \sqrt{\bar{\alpha}_{t-1}} \hat{x}_0 + \sqrt{1 - \bar{\alpha}_{t-1} - \sigma_t^2} \epsilon_\theta(x_t, t) + \sigma_t z,$$

where $\hat{x}_0 = (x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta) / \sqrt{\bar{\alpha}_t}$, and

$$\sigma_t(\eta) = \eta \sqrt{\frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}} \sqrt{1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}}.$$

When $\eta = 0$, the process becomes deterministic and can use far fewer steps (e.g., stride 4 on a 200-step schedule) without retraining. A 1D example demonstrates that DDIM with $\eta = 0$ and only 50 steps produces a sample distribution very close to the full 200-step DDPM. This makes DDIM a powerful sampling acceleration technique.

3.16 2D Blob Denoising with a Tiny U-Net

Beyond 1D, the chapter implements a full 2D DDPM on synthetic blob images (32×32 grayscale images with one or two random Gaussian blobs). A tiny U-Net (TinyUNet2D) with sinusoidal time embeddings is trained to predict noise. Denoising trajectories are shown: from pure noise, the reverse chain gradually reveals the blob shapes, and the final generated blobs closely resemble the training data. This example demonstrates how the U-Net architecture and time conditioning work together in a practical image diffusion model.

3.17 Common Diffusion Families

The chapter gives an overview of the broader diffusion landscape:

- **DDPM** – Canonical Gaussian Markov chain with noise-prediction training; stochastic sampling.
- **DDIM** – Implicit diffusion; same model, deterministic and/or fewer steps.
- **Score-SDE** – Continuous-time formulation; learns the score $\nabla_x \log p_t(x)$ and uses SDE/ODE solvers.
- **VE / VP** – Variance-Exploding (VE) vs. Variance-Preserving (VP) schedules; DDPM is VP.
- **Latent Diffusion (LDM)** – Diffuses in a compressed latent space, reducing compute (e.g., Stable Diffusion).
- **Guided Diffusion** – Incorporates class information via a classifier or classifier-free guidance.
- **Consistency / Flow Matching** – Very fast, few-step generation.

4 Chapter 4: Advanced Diffusion

4.1 Classifier-Guided Diffusion

While unconditional diffusion models generate images from random noise, we often want to steer the generation towards a specific class (e.g., “digit 3” or “cat”). Classifier-guided diffusion achieves this by training a separate noise-aware classifier $p_\phi(y \mid x_t, t)$ on noisy images at various timesteps. During sampling, the reverse process is modified by adding the gradient of the log-probability of the target class. Concretely, the guided score becomes $s_{\text{guided}}(x_t) = s_\theta(x_t) + w \nabla_{x_t} \log p_\phi(y \mid x_t, t)$, where w is the guidance scale. A larger w forces the sample to adhere more strongly to the class, but may reduce diversity and introduce artefacts. The classifier must be trained on noisy images with time conditioning to provide accurate gradients across the entire noise range.

4.2 Hyperparameters in Classifier-Guided Diffusion

Classifier-guided generation depends on several groups of hyperparameters. The **guidance scale** w is the most sensitive: small w gives weak conditioning but high diversity; large w enforces the class but can cause instability or artefacts. The **number of reverse steps** T affects both quality and conditioning; too few steps reduce fidelity. The **noise schedule** determines the difficulty of denoising and how well the classifier can distinguish classes at high noise levels. The **diffusion model capacity** must be sufficient; a weak unconditional model cannot be rescued by guidance. Similarly, the **classifier quality** is essential – it must be accurate on noisy inputs at

all timesteps, and **time conditioning in the classifier** is mandatory. **Timestep sampling** during classifier training (usually uniform) influences classifier performance at different noise levels. **Learning rate**, **batch size**, and **data normalisation** affect training stability and final accuracy. The **gradient location** (whether to guide on x_t directly or on a denoised proxy $\hat{x}_0$) can change stability. **Time scaling of classifier gradients** (e.g., multiplying by $\sqrt{1 - \bar{\alpha}_t}$) is sometimes used to avoid overpowering early denoising steps. A rule of thumb: first ensure the unconditional diffusion model is strong, then train a good noise-aware classifier, and finally tune w .

4.3 Classifier-Free Guidance (CFG)

Classifier-free guidance avoids training a separate classifier by using a single neural network $\epsilon_\theta(x_t, t, y)$ that can act both conditionally and unconditionally. During training, with some probability (e.g., 10–20%), the class label y is replaced by a null token $\emptyset$, forcing the model to learn both the conditional and unconditional data distribution. The training loss remains the standard MSE:

$$\mathcal{L}_{\text{CFG}} = \mathbb{E}_{x_0, t, \epsilon, y} \|\epsilon - \epsilon_\theta(x_t, t, y)\|^2.$$

At sampling time, the guided noise prediction is obtained by extrapolating between the conditional and unconditional predictions:

$$\hat{\epsilon}_\theta(x_t, t, y) = (1 + w) \epsilon_\theta(x_t, t, y) - w \epsilon_\theta(x_t, t, \emptyset),$$

where w is the guidance weight. This steers the generation towards y without any external classifier. CFG is widely used in large text-to-image models like Stable Diffusion because it elegantly controls the trade-off between fidelity to the condition and sample diversity.

4.4 SiLU (Swish) Activation Function

SiLU, also known as Swish, is defined by $\text{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$. It is a smooth, non-monotonic function that often outperforms ReLU in deep generative models including diffusion U-Nets. For large positive x , $\sigma(x) \approx 1$, so $\text{SiLU}(x) \approx x$; for large negative x , $\sigma(x) \approx 0$, so $\text{SiLU}(x) \approx 0$. Near zero it behaves like a soft gating of the identity. Its derivative is $\text{SiLU}'(x) = \sigma(x) + x \sigma(x)(1 - \sigma(x))$, which is smooth everywhere and can help avoid some optimization issues associated with activations that have kinks (like ReLU at zero).

5 Chapter: 5 Flow Matching

5.1 General Idea and Intuition Behind Flow Matching

Flow matching learns a time-dependent velocity field that transports samples from a simple base distribution (e.g., Gaussian noise) to a complex target distribution (the data). By defining a path—often a straight line—between paired base and data samples, the true instantaneous velocity along that path is known and used as a regression target. Training the model to predict this velocity creates a “wind field” that, when integrated via an ordinary differential equation (ODE), moves particles from noise to data samples.

5.2 How Flow Matching Obviates Estimation Issues Faced by VAE and Diffusion

Flow matching sidesteps two difficult estimation problems. Unlike diffusion models, it does not need to estimate the score function (the gradient of the log-density of noisy data), which can be sensitive in low-density regions. Unlike VAEs, it does not require learning an approximate

posterior distribution over latent variables, avoiding issues like posterior collapse or mismatched likelihood functions. Instead, flow matching converts generation into a supervised regression problem on explicit velocity targets derived from simple interpolation rules.

5.3 Intuition Behind Flow Matching

The process can be understood as learning a wind field. Imagine many balloons (particles) initially scattered according to a noise distribution; the goal is to blow them into the shape of the data distribution by time 1. For each balloon, a “training story” is created by picking a starting noise position, a target data position, and an intermediate time. The correct wind at that moment is the constant velocity that would move the balloon along the straight line between the two. The model learns to predict this wind from many such stories, and sampling simply follows the ODE to let the balloons travel along the learned wind.

5.4 Rationale for Various Neural Network Architectures in Flow Matching

The choice of architecture is driven by the data structure. For 1D problems, a multi-layer perceptron suffices. For images, U-Nets are preferred because their convolutional layers and skip connections capture spatial relationships, and time conditioning is injected via sinusoidal embeddings and feature-wise transformations. For 3D data or video, spatio-temporal backbones are used. The key requirement is that the network takes both the current state and time as input and outputs a vector field of the same dimensionality as the state.

5.5 Classifier-Guided and Classifier-Free Ideas Applied to Flow Matching

Classifier guidance incorporates an external classifier to steer generation towards a desired class by adding the gradient of the log-class probability to the velocity. Classifier-free guidance, on the other hand, jointly trains a conditional and an unconditional velocity model (by randomly dropping the label). At sampling time, a combination of the conditional and unconditional velocities is used, achieving controllability without a separate classifier and often yielding a better trade-off between sample quality and diversity.

5.6 Latent Flow Matching

To handle high-dimensional data like high-resolution images, flow matching can be performed in a lower-dimensional latent space learned by an autoencoder. The encoder compresses data into a compact representation, flow matching learns a velocity field on that latent space, and sampling is done by integrating the latent ODE and then decoding the result. This reduces computational cost and memory while maintaining the expressive power of flow matching, as the generative model focuses on the essential structure of the data.

5.7 Application of Optimal Transport to Flow Matching

Standard flow matching pairs base and data samples randomly, which can lead to crossing trajectories and curved average paths. Optimal transport (OT) instead finds a pairing that minimizes the total squared distance between the two sets, often within a minibatch. This yields straighter, shorter paths that are easier for the network to learn, improves training efficiency, and can enhance sample quality. Minibatch OT usually employs algorithms like the Hungarian method or Sinkhorn approximations.

5.8 Idea of Consistency Models

Consistency models build on the same ODE framework but train a single network to directly map any point along a probability flow trajectory to the trajectory’s endpoint (the data). The model is forced to satisfy a self-consistency condition: applying it to two nearby points along the trajectory should give the same result. This allows sampling with very few (even a single) network evaluations, dramatically accelerating generation compared to iterative ODE integration.

5.9 Why Consistency Models Are Difficult to Train

Enforcing the self-consistency property requires comparing the model’s output at two adjacent time steps, which introduces high-variance gradients and instability. The training loss is sensitive to the step size and discretization, and the model can collapse to trivial solutions if not carefully regularized. Achieving stable convergence often demands tailored network architectures, specialized training schedules, and painstaking hyperparameter tuning.

5.10 Design Dimensions and Variants of Flow Matching Solutions

Flow matching encompasses a family of generative models that differ along several axes: the probability path (e.g., straight-line, variance-preserving, or stochastic interpolants), the coupling between base and data samples (random, optimal transport, or minibatch OT), the neural network architecture (MLP, U-Net, transformer), the ODE solver (Euler, Heun, adaptive), the noise schedule, and whether to operate in data space or a latent space. Each choice influences training stability, sample quality, and computational cost.

5.11 Building Towards a Transport-Based View

Flow matching is naturally situated within the framework of optimal transport. The continuity equation links the velocity field to the evolution of the probability density, showing that the ODE transport map pushes the base distribution to the target. This perspective connects flow matching to the theory of measure-preserving flows, Wasserstein distances, and the idea of learning a ground-truth transport map from samples, providing theoretical grounding and opportunities to incorporate improvements from the optimal transport literature.

5.12 Some Publishable Ideas

Open research directions include developing better OT couplings that scale to large batches, exploring non-linear bridges that reduce trajectory curvature further, improving latent-space flow matching with more sophisticated autoencoders, designing novel guidance mechanisms for fine-grained control, accelerating sampling with advanced ODE solvers or consistency-like models, and extending flow matching to discrete data, molecule generation, or applications where theoretical guarantees on transport costs are required.

6 Chapter: 6 Transformers

6.1 What is an Autoregressive Model

An autoregressive model defines a probability distribution over a sequence by factorizing it into a product of conditional probabilities, each predicting the next token given all previous tokens. Formally, for a sequence $x_{1:T}$, we have $p(x_{1:T}) = \prod_{t=1}^T p(x_t | x_{1:t-1})$. In language modeling, the conditional distribution is typically categorical, obtained by applying softmax to a learned score vector. Training uses teacher forcing: at each step the ground-truth prefix is provided and the model is optimized to maximize the likelihood of the next token. Sampling (generation)

is autoregressive: the model feeds its own previous predictions as context to produce the next token, continuing until a stop condition is met.

6.2 How Recurrent Neural Networks (RNNs) Work

A recurrent neural network maintains a hidden state h_t that summarizes the past sequence. At each step, the current token x_t is embedded into a vector, and the hidden state is updated as $h_t = \phi(W_{xh}e(x_t) + W_{hh}h_{t-1} + b_h)$, where ϕ is an activation function such as $\tanh$. The same parameters are reused across all time steps. The updated hidden state is then mapped to output logits for next-token prediction. Training proceeds via backpropagation through time (BPTT), where gradients are propagated backward across the unrolled sequence. RNNs are natural for sequential data but can struggle with long-range dependencies because information must flow through many recurrent steps, leading to vanishing or exploding gradients.

6.3 Long Short-Term Memory (LSTM) and Its Limitations

LSTM extends the RNN by introducing a memory cell c_t and three gating mechanisms: input, forget, and output gates. The cell state provides a linear information highway across time, while the gates (each a sigmoid layer producing values between 0 and 1) learn when to write new information, erase old memory, or expose memory to the output. The hidden state is computed as $h_t = o_t \odot \tanh(c_t)$. This design greatly mitigates vanishing gradients and allows the network to retain information over long spans. However, LSTMs remain inherently sequential, making training slow on long sequences and still susceptible to gradient issues over very long distances. Moreover, the single hidden state must compress all contextual information, which can be a bottleneck for complex dependencies.

6.4 Basic Ideas Behind the Transformer Architecture

Transformers replace recurrence with self-attention, enabling direct interaction between any pair of tokens in a sequence. At each layer, every token constructs a query, key, and value vector through learned projections. Scaled dot-product attention computes a weighted combination of values, where the weights are determined by the similarity between the query and keys. Multi-head attention runs several attention operations in parallel, allowing the model to capture different types of relationships. Because self-attention is permutation-invariant, positional encodings are added to token embeddings to convey order. Each layer also includes a position-wise feed-forward network and residual connections with layer normalization. This design enables parallel computation across the sequence, much shorter gradient paths for long-range dependencies, and excellent scalability to large datasets and models.

6.5 Byte-Level BPE Tokenization

Byte-level Byte-Pair Encoding (BPE) constructs a vocabulary of subword tokens directly from raw UTF-8 bytes. Starting with the 256 individual byte values as the initial vocabulary, the algorithm iteratively counts adjacent symbol pairs in the training corpus, merges the most frequent pair into a new token, and repeats until a desired vocabulary size is reached. Because the base symbols are bytes, any Unicode text can be represented, eliminating out-of-vocabulary issues. After training, new text is tokenized deterministically by applying the learned merges in order. BPE balances the flexibility of character-level models with the efficiency of word-level models, producing short sequences with manageable vocabulary sizes and good coverage of rare words and morphology.

6.6 Decoder-Only Transformer with Causal Self-Attention

A decoder-only (GPT-style) Transformer generates text autoregressively by applying causal masking in self-attention. For a token at position t , the attention scores to future positions $j > t$ are set to $-\infty$ before the softmax, so the model can only condition on earlier tokens. This allows the model to be trained with teacher forcing on all positions in parallel while respecting the autoregressive property. Typically, pre-layer-norm blocks are used: the input is normalized before attention and the feed-forward network, with residual connections added afterwards. The final hidden states are projected back to the vocabulary size, and the model is trained with a next-token cross-entropy loss. During generation, tokens are sampled one by one using the causal property, often with temperature scaling and top- k sampling for quality control.

6.7 Training, Evaluation, and Generation on a Simple Corpus

To train a small decoder-only Transformer on a text corpus like Tiny Shakespeare, the text is first tokenized into a sequence of integer IDs. Training samples are constructed by sliding a context window across the tokenized data: a block of consecutive tokens serves as input x , and the same block shifted by one serves as the target y . The model processes x and produces logits for each position, and cross-entropy loss is computed between these logits and the true next tokens in y . A validation set is used to monitor perplexity without dropout. After training, text generation starts from a prompt; the model repeatedly feeds the last ‘block_size’ tokens, obtains next-token probabilities (possibly scaled by temperature), and samples from the top- k most likely tokens to produce coherent continuations.

6.8 Worked Attention Example: Scores, Masking, and Softmax Weights

A concrete walk-through of single-head self-attention clarifies the mechanism. Given a small input matrix of token embeddings X , learned projection matrices W_Q, W_K, W_V produce queries, keys, and values. The raw attention scores are computed as $S = QK^\top / \sqrt{d_k}$. For autoregressive models, a causal mask sets upper triangular entries to $-\infty$, preventing future tokens from being attended to. A row-wise softmax converts the masked scores into non-negative attention weights that sum to one. The output for each token is a weighted sum of the value vectors from all allowed positions: $Y = \text{softmax}(S_{\text{masked}})V$. This operation gives each token a context-dependent representation that selectively incorporates information from past tokens according to the learned attention pattern.

7 Chapter 7: Vision Transformers

7.1 Adapting Transformers to Vision

The Transformer architecture, originally designed for natural language processing, processes sequences of token embeddings through self-attention. To apply it to images, the core challenge is to convert a 2D image into a 1D sequence of tokens while preserving spatial information. Vision Transformers (ViT) solve this by partitioning the image into fixed-size patches, linearly embedding each patch, and treating these embeddings as input tokens. The resulting model inherits the ability to model long-range dependencies through global self-attention, directly linking every patch to every other patch from the first layer onward.

7.2 Patch Embedding: From Image to Sequence

Given an input image of size $H \times W \times C$, ViT divides it into a grid of $P \times P$ patches. The number of patches is $N = \frac{HW}{P^2}$. Each patch is flattened into a vector of length P^2C and projected to a fixed dimension D via a trainable linear layer (or equivalently, a convolution with stride equal

to the patch size). This produces a sequence of N patch embeddings. This sequence is then prepended by a special learnable classification token, and position embeddings are added before being fed into the Transformer encoder.

7.3 Position Embeddings in ViT

Self-attention is permutation-invariant, so position information must be explicitly supplied. ViT adds a learnable $1D$ position embedding to each patch token (and the classification token). These embeddings are vectors of dimension D , one per spatial position. After training, they encode spatial structure; nearby patches often have similar position embeddings. Alternative schemes like $2D$ sinusoidal embeddings or relative position biases are sometimes used, but standard ViT demonstrates that simple learned positional encodings suffice to capture spatial arrangement when trained on sufficiently large datasets.

7.4 The Classification Token (CLS Token)

Inspired by BERT, ViT prepends a special learnable embedding x_{class} to the sequence of patch embeddings. This token is not derived from any image patch; its role is to aggregate information from the entire image through the self-attention layers. After passing through the Transformer encoder, the final hidden state corresponding to the classification token is used as the image representation for classification. This design avoids committing to a particular pooling strategy and allows the model to dynamically gather global context.

7.5 Multi-Head Self-Attention in Vision

Inside a ViT encoder block, multi-head self-attention (MHSA) is applied identically to NLP Transformers. Each patch token attends to every other patch token (plus the class token) with full bidirectional context. This gives the model a global receptive field from the very first attention layer, in contrast to the local, hierarchical receptive fields of convolutional networks. The attention mechanism computes relationships between all pairs of patches, enabling the model to integrate long-range spatial cues immediately.

7.6 ViT Architecture Overview

A standard ViT consists of a stack of L identical Transformer encoder blocks. Each block contains:

- LayerNorm $\rightarrow$ Multi-Head Self-Attention $\rightarrow$ Residual connection.
- LayerNorm $\rightarrow$ MLP (two linear layers with GELU activation) $\rightarrow$ Residual connection.

The MLP typically expands the hidden dimension to $4D$ and then projects back to D . The classification head is a single linear layer on top of the final class token embedding. The model uses pre-layer normalisation, the same as modern GPT-style transformers but with full (non-causal) attention.

7.7 Comparison with CNNs: Inductive Biases

Convolutional neural networks (CNNs) possess strong inductive biases: locality (each neuron only connects to a local region), translation equivariance (same kernel applied everywhere), and a hierarchical structure built by pooling/striding. ViT, by contrast, imposes virtually no prior about the spatial structure of images. Self-attention is global and isotropic; all patches interact equally regardless of distance. This absence of built-in biases makes ViT highly flexible but also data-hungry – it needs massive datasets (or strong data augmentation and regularisation) to learn that nearby patches are more correlated than far-away ones. Once trained on sufficient

data, ViT often surpasses CNNs, especially on large-scale tasks, because it can learn the most useful spatial configurations from data rather than having them hard-coded.

7.8 Training and Data Requirements

Because ViT lacks the locality and translation equivariance priors of CNNs, it underfits severely when trained only on mid-sized datasets like ImageNet-1K with no pretraining. Effective training relies on large-scale pre-training (e.g., ImageNet-21K or JFT-300M) followed by fine-tuning, or on strong regularisation and distillation (as in DeiT). Key training elements include:

- Large batch sizes and high learning rates.
- Extensive data augmentation (RandAugment, MixUp, CutMix).
- Regularisation via weight decay, label smoothing, and stochastic depth.
- Transfer learning from pre-trained models on huge datasets.

With these, ViT can match or exceed state-of-the-art CNNs.

7.9 Variants and Extensions

Since the original ViT, several important variants have been developed:

- **DeiT (Data-efficient Image Transformer)** – Uses teacher-student distillation and strong augmentations to train ViT on ImageNet-1K from scratch.
- **Swin Transformer** – Introduces shifted-window attention and hierarchical patch merging, rebuilding multi-scale feature pyramids and restoring a locality bias, achieving excellent performance on dense prediction tasks (detection, segmentation).
- **PVT (Pyramid Vision Transformer)** – Gradually reduces spatial resolution and increases channel depth through patch merging, similar to CNN backbones.
- **Cross-ViT, LeViT, T2T-ViT** explore different tokenisation, efficiency improvements, or hybrid CNN-Transformer designs.

These extensions address ViT’s original limitations (lack of multi-scale features, quadratic complexity of full attention for high-resolution images) and extend its applicability to a broader range of vision tasks.

7.10 Summary

Vision Transformers demonstrate that a pure self-attention architecture, with minimal vision-specific inductive biases, can achieve state-of-the-art performance on image classification and beyond, provided sufficient data and training resources. The key design decisions – patch granularity, position encoding, the classification token, and global self-attention – define the original ViT. Subsequent variants reintroduce some inductive biases (locality, hierarchy, shifted windows) to improve data efficiency, speed, and performance on dense tasks, making the ViT family a dominant paradigm alongside CNNs in modern computer vision.

8 Chapter 8: Vision–Language Models with a Special Emphasis on CLIP

8.1 Dual-Encoder Contrastive (CLIP-like) Architecture

The dual-encoder family uses two separate neural networks: an image encoder and a text encoder. Each maps its input to a fixed-dimensional embedding vector. The networks are trained so that embeddings of matching image–text pairs are close (e.g., via cosine similarity), while embeddings of non-matching pairs are far apart. The objective is a contrastive loss computed over a batch of paired examples; for a batch of size N , the model computes an $N \times N$ similarity matrix and applies a symmetric cross-entropy loss that pulls matched pairs together and pushes

all others apart. Because the two encoders produce embeddings in a shared space, the model naturally supports retrieval (find the nearest image for a text query or vice versa) and zero-shot classification (compare an image embedding to text embeddings of class descriptions).

8.2 Fusion / Encoder–Decoder Architecture and Cross-Attention

In contrast to dual encoders, fusion models deeply intertwine visual and textual information. They typically employ an image encoder that produces a sequence of visual tokens (e.g., patch features from a ViT or flattened CNN feature maps). A text decoder then generates output tokens autoregressively. At each decoder layer, masked self-attention processes the text, and then cross-attention lets each text token query the visual tokens. The visual tokens act as a memory: the text queries attend over all image positions, mixing relevant visual information into the text representation. This design enables strong conditional text generation, making it suitable for captioning, visual question answering, and instruction following.

8.3 Role of Paired Image–Text Data and Data Noise

Large-scale vision–language models are trained on enormous collections of image–caption pairs, often harvested from the web. The quality of these pairs strongly influences model performance. Noisy or weakly aligned captions (e.g., automatically extracted alt-text) can still be useful when the dataset is massive, as demonstrated by models like ALIGN. However, extremely mismatched pairs degrade the learned alignment. Modern training pipelines therefore incorporate data filtering (e.g., by CLIP score) and careful curation. The paired data define the concepts the model can distinguish; biases, gaps, and geographical/cultural skews in the training data propagate into the model’s behaviour and must be evaluated separately.

8.4 Evaluation of VLMs: Beyond Single Benchmarks

Evaluating vision–language models requires a multidimensional view beyond a single accuracy number. Typical desiderata include:

- **Retrieval metrics:** $\text{recall}@k$ (e.g., $\text{recall}@1$, $\text{recall}@5$) for both image-to-text and text-to-image retrieval, measuring how often the correct counterpart appears among the top- k results.
- **Zero-shot classification accuracy:** the ability to recognise unseen classes from textual descriptions without fine-tuning.
- **Robustness:** performance under distribution shifts, domain transfer, and adversarial perturbations.
- **Bias and fairness:** measurement of performance disparities across demographic groups, geographic regions, or rare concepts, including stereotypical associations.
- **Efficiency:** latency, memory footprint, and energy consumption, especially important for deployment on edge devices or large-scale serving.

A thorough evaluation considers all these axes, often using suites like the CLIP Benchmark or various retrieval and VQA datasets.

8.5 The CLIP Similarity Matrix and Contrastive Loss

At the core of CLIP-style training is a batch-level contrastive loss. Given a batch of N (image, text) pairs, the image encoder produces N normalised embeddings v_i and the text encoder produces N normalised embeddings t_i . The cosine similarity matrix is $S = VT^\top$, where V and T are matrices whose rows are the v_i and t_i vectors. A temperature parameter τ scales these logits: $\text{logits} = S/\tau$. The loss treats the N correct pairs as positive classes and all other $N^2 - N$ pairings as negatives. It consists of two cross-entropy terms: one for image-to-text matching

(each row of the logits matrix should have the highest score at the diagonal) and one for text-to-image matching. This symmetric objective encourages aligned pairs to have high similarity while repelling unaligned ones.

8.6 Zero-Shot Classification with CLIP

Because the dual-encoder produces aligned embeddings, CLIP can perform zero-shot classification by converting class labels into natural language prompts. For each class, a textual description (e.g., “a photo of a cat”) is encoded into a text embedding w_c . An image embedding v is compared to all class prompts via dot product: $\hat{c} = \arg \max_c v^\top w_c$. The temperature learned during training is not needed for the argmax. This approach lets the model recognise categories never seen during training, provided the class names can be described in natural language, and has been widely used as a strong baseline for open-vocabulary recognition.

8.7 Cross-Modal Retrieval in CLIP

Once the joint embedding space is trained, retrieval in either direction is straightforward. For image-to-text retrieval, one computes the similarities between a query image embedding and all candidate text embeddings, then ranks texts by decreasing similarity. For text-to-image retrieval, the process is reversed. The top- k recall metrics directly measure whether the true paired item appears among the k most similar items. The embedding-based retrieval is highly efficient because the full set of embeddings can be pre-computed and indexed, enabling real-time search even on large databases.

8.8 Joint Embedding Space and t-SNE Visualization

The training process shapes a shared latent space where matched image and text pairs lie close together. Visualising this space with dimensionality reduction (e.g., t-SNE) reveals clear clustering of concepts. When images and their corresponding captions are projected, they often occupy overlapping regions. When one encodes a fixed set of class prompts (like “a photo of the digit X”), the prompt embeddings tend to fall near the centre of the image clusters for that class. This geometric view directly explains why zero-shot classification works: the nearest class prompt in embedding space corresponds to the correct label.

8.9 Image-Conditioned Text Generation in Fusion Models

Fusion-based vision-language models treat the image as context and generate text autoregressively. Formally, for an image x and a target caption token sequence $y = (y_1, \dots, y_T)$, the model maximises $\sum_{t=1}^T \log p_\psi(y_t \mid y_{<t}, x)$ via teacher forcing. The image is first encoded into a sequence of visual tokens (the encoder memory). A Transformer decoder then processes the caption tokens with causal self-attention and cross-attends to the visual memory at each layer. The final layer’s hidden state for the last token is used to predict the next token. At inference, generation proceeds token by token, using the previously sampled tokens as context.

8.10 Cross-Attention Mechanism in Fusion VLMs

Cross-attention is the central operation that fuses visual and linguistic information. At a given decoder layer, let the hidden text representations be H . Queries are computed from H , while keys and values are computed from the visual memory M . The cross-attention weights are obtained by a scaled dot product between queries and keys, normalised over the visual tokens via softmax. Each text token thus produces a weighted sum of visual value vectors, effectively retrieving information from all image regions. A residual connection adds the result back to the text stream, and a subsequent feed-forward network refines the representation. Using multiple

heads allows different attention patterns (e.g., focusing on shape, text, or objects). All projection matrices for queries, keys, values, and the output are learnable.

8.11 Extensions Beyond CLIP

The CLIP framework has inspired a large ecosystem of models and applications. Key extensions include:

- **Improved training objectives:** SigLIP replaces the softmax-based contrastive loss with a pairwise sigmoid loss, which scales better to large batch sizes and multilinguality.
- **Prompt learning:** Instead of hand-crafting prompt templates, methods like CoOp and CoCoOp learn continuous prompt vectors in the embedding space, improving few-shot and zero-shot performance.
- **Lightweight adaptation:** Adapters (e.g., CLIP-Adapter, Tip-Adapter) introduce small trainable modules to adapt a frozen CLIP backbone for new tasks without fine-tuning the whole model.
- **Efficient and tiny CLIP models:** Distillation and pruning produce compact models (TinyCLIP, MobileCLIP) that retain much of the original performance with drastically reduced compute.
- **Dense prediction:** Works like CLIPSeg and DenseCLIP adapt CLIP for pixel-level tasks such as open-vocabulary semantic segmentation.
- **Object detection and grounding:** OWL-ViT, GLIP, and Grounding DINO extend the dual-encoder idea to localise objects described by free-form text.
- **Fusion and multimodal LLMs:** Flamingo, BLIP-2, and LLaVA combine a pretrained vision encoder with a large language model via a learnable bridge or direct projection, enabling rich multimodal dialogue and reasoning while leveraging strong text-only capabilities.

These extensions collectively demonstrate that CLIP’s core idea—a shared image–text embedding space—is a versatile foundation that can be adapted to nearly all vision–language tasks.

9 Chapter 9: Training and Adapting Vision-Language Models

9.1 The Adaptation Ladder for VLMs

Adapting a pre-trained vision-language model (VLM) can be viewed as a ladder of methods, ranging from zero-shot prompting (no parameter changes, only input modification) through few-shot in-context learning, retrieval-augmented generation (RAG), supervised fine-tuning (SFT), parameter-efficient fine-tuning (PEFT, e.g., LoRA/QLoRA), preference optimisation (DPO), and finally online reinforcement learning (RLHF/PPO/RLOO). Methods at the lower rungs do not alter model weights but modify the prompt or the provided context; higher rungs update model parameters, from a subset of weights (adapters) to full policy optimisation. This ladder offers a systematic trade-off between cost, data requirements, and the degree of specialisation achievable for a downstream task.

9.2 Weight-Updating vs. Prompt-Only Methods

Methods are split into those that keep model weights frozen and those that update them. Zero-shot prompting, few-shot exemplars, and most RAG setups change only the model’s input context. They are quick, require no training, but offer limited customisation. In contrast, SFT, LoRA, QLoRA, DPO, and RL-based methods (PPO, RLOO) modify the conditional distribution $p_{\theta}(y \mid x, v)$ by adjusting model parameters. Weight-updating methods yield stronger, more persistent adaptation at the cost of collecting supervision data and performing gradient-based training.

9.3 Multimodal Input Processing in a VLM Pipeline

A modern VLM receives both text and images as input. The text prompt (instructions, class candidates, context) is tokenised into a sequence of tokens. Images are processed by a vision encoder, often producing a set of visual tokens (patch features). Special tokens (e.g., ‘<image>’) mark where to inject visual tokens. A processor (e.g., HuggingFace AutoProcessor) convolves these modalities into a single sequence of embeddings and applies a chat template to generate the model’s input. The model then outputs structured text, which may be parsed to extract fields like scientific names or confidence scores. This pipeline allows flexible zero-shot and few-shot interaction without architectural changes.

9.4 Few-Shot Learning and RAG Without Weight Updates

Few-shot learning provides a small number of input-output exemplars in the prompt, steering the model’s format, style, and task behaviour. Because parameters are unchanged, the model relies entirely on its pre-trained knowledge and the demonstration context. Retrieval-Augmented Generation (RAG) goes a step further: it fetches external knowledge (e.g., species descriptions, field guide pages) and injects it into the prompt. This approach is particularly useful for grounding answers in verifiable sources, improving accuracy on knowledge-intensive tasks without fine-tuning. Both methods are cheap, require no gradient computation, and can be combined.

9.5 Supervised Fine-Tuning (SFT) Objective

SFT updates model weights to maximise the likelihood of a target response given the image and prompt. For a dataset of examples $\{(x_i, v_i, y_i)\}$, where y_i is the desired answer, the loss is the negative log-likelihood of the answer tokens under the model’s autoregressive distribution:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(y_{i,t} \mid y_{i,<t}, x_i, v_i).$$

This directly teaches the model to produce a specific output format and content, making it appropriate for tasks where high-quality annotated examples are available.

9.6 LoRA and QLoRA: Parameter-Efficient Fine-Tuning

Low-Rank Adaptation (LoRA) freezes the pre-trained weights and inserts trainable low-rank matrices into the linear layers of the model. For a weight matrix $W \in \mathbb{R}^{d \times k}$, LoRA parameterises the update as $W + \alpha \cdot AB$, where $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ with rank $r \ll \min(d, k)$. This drastically reduces the number of trainable parameters. QLoRA builds on LoRA by quantising the frozen base model to 4-bit precision (NF4), further cutting memory usage and enabling fine-tuning on a single powerful GPU. In practice, adapters are applied to the query and value projection matrices (and often also the output and gate projections) of the Transformer’s attention layers. Both techniques deliver much of the benefit of full fine-tuning at a fraction of the compute cost.

9.7 Preference Data for DPO

Direct Preference Optimisation (DPO) operates on triples of the form (prompt + image, chosen response, rejected response). The chosen answer is preferred over the rejected one, e.g., because it contains the correct scientific name and appropriate confidence. These pairs do not need to be perfect; they only need to express a relative preference. DPO directly optimises the policy to increase the log-probability of the chosen relative to the rejected, while remaining close to a reference model.

9.8 DPO Loss and Intuition

Given a reference model π_{ref} , the DPO loss for a preference pair (y_w, y_l) is

$$\mathcal{L}_{\text{DPO}}(\theta) = -\log \sigma\left(\beta\left[\log \pi_{\theta}(y_w|x, v) - \log \pi_{\theta}(y_l|x, v) - \log \pi_{\text{ref}}(y_w|x, v) + \log \pi_{\text{ref}}(y_l|x, v)\right]\right),$$

where β controls the strength of the preference update. The loss encourages the policy to prefer y_w over y_l more strongly than the reference does, effectively performing a smooth, stable form of reward-weighted optimisation without fitting an explicit reward model. DPO is an offline method: it uses a fixed dataset of preferences, avoiding costly online sampling during training.

9.9 KTO and Alternatives

KTO (Kahneman-Tversky Optimisation) is another preference-based alignment method that can work with unpaired or only-chosen/only-rejected data. It often has a simpler implementation and can be more robust when constructing clear rejections is difficult. Both DPO and KTO convert human (or synthetic) preferences into a supervised-style learning signal without the complexity of full reinforcement learning.

9.10 Online Reinforcement Learning for VLM Alignment

Online RL methods, such as PPO and RLOO, actively sample new completions from the current policy during training. A reward model (or heuristic reward function) scores these completions. The policy is then updated to increase the probability of high-reward outputs, subject to a KL penalty that prevents the policy from drifting too far from a reference model. PPO uses a clipped surrogate objective and advantage estimation; RLOO simplifies the process by generating multiple completions per prompt and using the leave-one-out baseline:

$$A_i = r_i - \frac{1}{G-1} \sum_{j \neq i} r_j,$$

where r_i is the reward of the i -th completion. The policy is then updated via a REINFORCE-style gradient scaled by A_i . Online RL is the most potent and flexible alignment stage, but it is also the most computationally demanding and requires careful reward design to avoid reward hacking.

9.11 Key Reinforcement Learning Concepts in VLM Alignment

The VLM’s next-token distribution acts as a policy $\pi_{\theta}(a_t | s_t)$, where the state s_t includes the image, prompt, and prefix. A reward function assigns a scalar score to a completed answer. The return is the cumulative reward. The advantage A_t compares the reward of a particular action (or sequence) to a baseline, helping reduce variance. Baselines can be learned value functions or, as in RLOO, the mean reward of other completions. KL regularisation penalises large deviations from the reference policy, preserving general capabilities. These concepts form the mathematical backbone of RL-based VLM alignment.

9.12 Trade-Offs in Choosing an Adaptation Method

The adaptation ladder embodies a clear set of trade-offs. Zero-shot and few-shot require no training data or GPU hours but offer minimal improvement. RAG adds retrieval cost and can greatly enhance grounding, but still does not change the model. SFT yields a strong, specialised model but needs relatively many labelled examples and substantial GPU time. LoRA/QLoRA provide a cheaper entry point to weight-updating methods, often with comparable quality. DPO refines alignment with preference data but requires constructing high-quality preference pairs. Finally, online RL (PPO/RLOO) gives the greatest fine-grained control but demands the most

compute, engineering effort, and reward design expertise. The choice thus depends on available labelled data, hardware budget, the desired level of specialisation, and the acceptable risk of over-fitting or reward hacking.

10 Chapter 10: Quantization Techniques for Deep Learning and Generative Models

10.1 Motivation for Quantization

Quantization replaces high-precision numerical representations (e.g., FP32) with lower-precision integer formats. The primary motivations are reducing memory footprint, memory bandwidth, arithmetic throughput, and energy consumption, making large models deployable on resource-constrained hardware. If a model’s parameters are stored in b bits each, the total storage scales linearly with b . Moving from FP32 to INT8 reduces parameter storage by a factor of four; INT4 yields an eightfold reduction. Quantization also accelerates inference when dedicated low-precision kernels are used, but only if dequantization overhead is controlled, activations are also quantized compatibly, and numerical outliers do not force fallback to high precision.

10.2 Three Phases of Quantization

Quantization is studied in three phases that correspond to different points in a model’s lifecycle. **Pre-quantization** analyses the numerical properties of a trained model—outliers, dynamic ranges, per-layer sensitivity—to decide bit-widths and granularity before any low-precision conversion. **During-training quantization (QAT)** incorporates quantization into the optimisation loop, so the model learns parameters that are robust to rounding, clipping, and reduced resolution. **Post-training quantization (PTQ)** starts from a high-precision trained model and finds the best low-precision surrogate with limited or no retraining. The three phases are complementary perspectives on the same core challenge: how to preserve task quality while drastically lowering numerical precision.

10.3 Standard Quantization Formula and Parameters

Uniform affine quantization maps a real value x to an integer q via a scale $s > 0$ and zero-point z :

$$q = \text{clip}\left(\text{round}\left(\frac{x}{s}\right) + z, q_{\min}, q_{\max}\right),$$

and reconstructs $\hat{x} = s(q - z)$. The integer range $[q_{\min}, q_{\max}]$ is determined by the bit-width b ; for signed symmetric b -bit quantization it is typically $[-(2^{b-1} - 1), 2^{b-1} - 1]$. The scale s sets the distance between adjacent representable levels, and the zero-point z anchors the integer grid to the real values. In symmetric quantization $z = 0$, simplifying arithmetic.

10.4 The Scale–Range Trade-off

For a fixed bit-width, the scale s controls both resolution and range. A smaller s gives finer spacing and lower rounding error inside the range, but the representable interval $[s q_{\min}, s q_{\max}]$ becomes narrow, causing severe clipping of larger values. A larger s widens the range and reduces clipping, but coarsens the grid and increases rounding error everywhere. Thus choosing the scale is fundamentally a trade-off between two distortion sources: granularity error from rounding, and saturation error from clipping. Much of quantization research—calibration rules, learned thresholds, mixed precision—aims to balance this tension.

10.5 Per-Tensor, Per-Channel, and Group-wise Quantization

Quantization granularity determines how many scales are used. **Per-tensor** quantization assigns one scale to an entire weight tensor; it is simple but fails when different rows or columns have very different dynamic ranges. **Per-channel** quantization uses a separate scale for each output channel (each row of a weight matrix), greatly reducing error in transformers and CNNs where channel magnitudes can vary widely. **Group-wise** quantization partitions channels into groups and assigns a scale per group, providing a flexible intermediate between per-tensor and per-channel that is often more hardware-friendly than full per-channel.

10.6 Quantization-Aware Training (QAT)

In QAT, quantization is embedded in the forward pass using **fake quantization**, where weights and activations are quantized and then dequantized before the next operation, but gradients flow through a straight-through estimator (STE) that treats the rounding operation as identity for backward passes. The training objective explicitly accounts for layerwise quantizers:

$$\min_{\{W_\ell\}, \{\theta_\ell\}} \mathcal{L}(f(x; \{Q_W(W_\ell; \theta_\ell)\}, \{Q_a(a_\ell; \theta_\ell)\})).$$

This allows the model to learn parameters that are inherently robust to low-precision distortion. Methods such as PACT and LSQ further learn the clipping thresholds or step sizes themselves, enabling aggressive bit-widths (3–4 bits) with minimal accuracy loss.

10.7 Post-Training Quantization (PTQ)

PTQ converts an already trained floating-point model to low precision without full retraining. The basic approach is round-to-nearest (RTN), but simple rounding often degrades performance at low bits. More sophisticated PTQ methods proceed along a spectrum: **AdaRound** learns optimal rounding offsets; **second-order methods** (e.g., GPTQ) use the Hessian of the loss to prioritise weight perturbations that least affect task loss; **SmoothQuant** migrates activation outliers into the weights by a diagonal rescaling, making activations easier to quantize; **AWQ** protects salient channels identified by activation magnitudes; **OmniQuant** jointly optimises scaling, rounding, and other parameters in a unified framework. PTQ is critical for large pre-trained models where full retraining is infeasible.

10.8 Second-Order Sensitivity and the Role of the Hessian

A second-order Taylor expansion of the task loss around the full-precision weights W gives, for a small perturbation $\Delta W = \widehat{W} - W$,

$$\mathcal{L}(\widehat{W}) \approx \mathcal{L}(W) + \nabla \mathcal{L}(W)^\top \Delta W + \frac{1}{2} \Delta W^\top H \Delta W.$$

At a trained optimum the gradient term is negligible, so the loss increase is dominated by the quadratic form $\frac{1}{2} \Delta W^\top H \Delta W$. This shows that not all weight errors are equally harmful; errors in directions of high Hessian curvature cause larger loss increases. GPTQ and other second-order PTQ methods use an approximation of the inverse Hessian to sequentially quantise columns while compensating remaining weights, effectively keeping the quadratic penalty small.

10.9 Modern Transformer and LLM Quantization Methods

Several specialised approaches address the unique challenges of large transformers:

- **LLM.int8()** handles outlier features by using mixed precision: vector-wise quantization for most values and FP16 for extreme outliers.

- **GPTQ** applies second-order column-wise compensation to quantise large language models to 3–4 bits with minimal performance loss.
- **SmoothQuant** rescales activations by a per-channel factor and the weights by the inverse factor, so that activation outliers are tamed without changing the model output.
- **AWQ** observes that only a tiny fraction (1%) of salient weight channels need protection; it computes per-channel scaling to preserve them.
- **OmniQuant** jointly learns clipped ranges, rounding parameters, and scaling factors in a differentiable way.
- **I-BERT** redesigns the entire inference graph to operate with integer-only arithmetic, including approximations for GELU, softmax, and LayerNorm.
- **QLoRA** enables fine-tuning a 4-bit quantised base model by adding low-rank adapters, combining memory efficiency with task adaptation.

10.10 Quantization for Generative Models

Generative models (LLMs, diffusion models, flow-matching models) pose extra difficulties because they are used iteratively: autoregressive decoding, repeated denoising steps, or ODE integration. Quantisation error can accumulate over time, degrading output quality. For LLMs, attention projections, KV caches, and activation outliers are critical. For diffusion models, activation statistics vary strongly with the timestep, and small per-step errors compound across many denoising steps; methods like Q-Diffusion and PTQD introduce timestep-aware calibration and noise-compensation strategies. For flow-matching and continuous-time models, the quantised velocity field $\hat{v}_\theta(x, t) = v_\theta(x, t) + e(x, t)$ perturbs the ODE $d\hat{x}_t = \hat{v}_\theta(\hat{x}_t, t) dt$, and even small errors can alter the integrated trajectory. The field of flow-matching quantisation is still emerging, borrowing ideas from transformer PTQ and diffusion-aware design.

10.11 Comparative Challenges Across Model Families

Quantisation difficulty varies by model architecture and modality. **LLMs** suffer from activation outliers, KV-cache precision, and cross-layer error propagation. **Diffusion models** add timestep-dependent statistics and iterative error accumulation. **Flow-matching models** require preserving the quality of a continuous-time transport path. **CLIP-like dual encoders** must maintain cross-modal alignment in a shared embedding space; quantisation that widens the image–text similarity gap can degrade zero-shot performance. **Multimodal fusion models (LVLMs)** face modality imbalance, where vision tokens, language tokens, and cross-attention blocks have very different sensitivities, demanding modality-aware and token-saliency-aware PTQ. Across all families, an effective strategy often involves mixed precision, calibration on representative data, and borrowing principles from curvature- and saliency-aware method.